PROGRAMMATION WEB DYNAMIQUE

Benoit Gaudou - benoit.gaudou@ut-capitole.fr

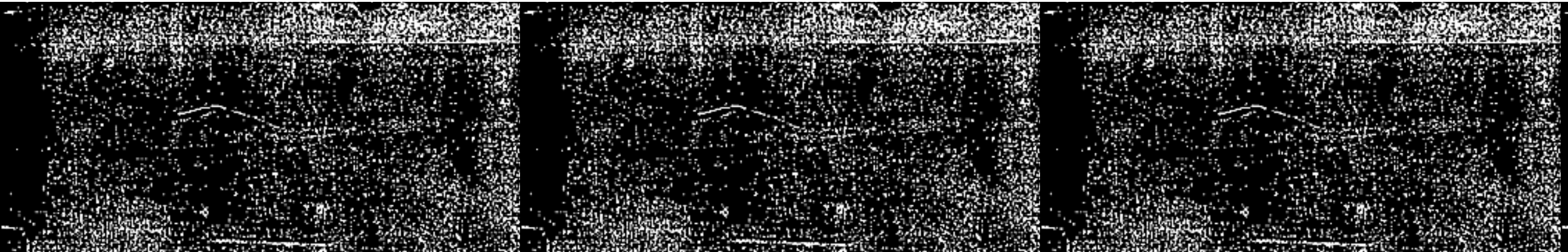
Source : Laurent Perrussel

动态网络程序设计

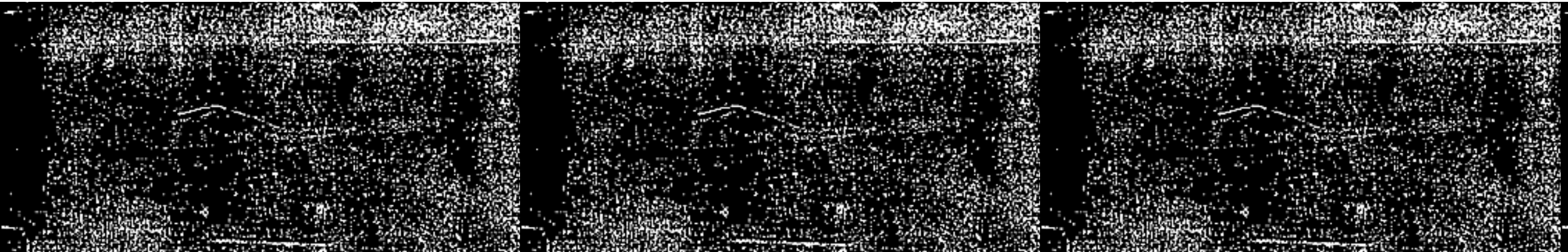
本诺·高杜 -benoit.gaudou@ut-capitole.fr

来源：Laurent Perrussel

TEMPLATING



TEMPLATING



MOTEUR DE TEMPLATE

- Spring (boot) se base sur des moteurs de template pour générer les pages HTML. Le moteurs supportés incluent :
  - JSP
  - FreeMarker
  - Thymeleaf

- Un fichier de template **contiendra des instructions pour générer une page HTML**, **à partir de données** transmises depuis la méthode gérant la route associée.
- En Thymeleaf, the fichier de template est un fichier **HTML valide** : les **instructions sont des attributs de balises HTML**.

MOTEUR DE TEMPLATE

- Spring（靴子）基于模板引擎生成HTML页面。支持的引擎包括：
  - JSP
  - 游离标记
  - 百里香叶
- 模板文件将包含生成HTML页面的指令，这些指令基于路由管理方法传递的数据。
- 在Thymeleaf中，模板文件是有效的HTML文件：其中的指令均为HTML标签属性。

EXEMPLE DE TEMPLATE

greetings.html

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
</html>
```

EXEMPLE DE TEMPLATE

问候

```
! DOCTYPE HTML>
<html xmlns:th= “http://www.thymeleaf.org” >
< 头部 >
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
</html>
```

EXEMPLE DE TEMPLATE

greetings.html

```html
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
    <p th:text="|Hello, ${nameT}!| » />
</body>
</html>
```

```java
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("nameT", name);
        return "greetings";
    }
}
```

EXEMPLE DE TEMPLATE

问候

```
! DOCTYPE HTML>
<html xmlns:th= “http://www.thymeleaf.org” >
< 头部 >
    <title> 开始使用：Web 内容服务 </title>
    <meta http-equiv= “Content-Type” content= “ text/html;  charset=UTF-8”  /></head>
< 体>
    <p第:text= “ |你好，${nameT}！  |»/>
</体
</html>
```

```
控制台
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("nameT", name);
        return "greetings";
    }
}
```

EXEMPLE DE TEMPLATE

- **La transmission de données entre le contrôleur et le template se fait au travers de la variable `Model model`.**
- Des objets de tout type peuvent être ajoutés dans le model (int, String, List …)

```
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        return "greetings";
    }
}
```

greetings.html

```
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
```

EXEMPLE DE TEMPLATE

- 控制器与模板之间的数据传输通过Model model变量实现。
- 可向模型中添加各类对象（整数、字符串、列表等）

控制台

```
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        return "greetings";
    }
}
```

问候

```
< 体>
    <p th:text= “ | 你 好 ， ${name}! | ”  />
</body>
```

EXEMPLE DE TEMPLATE

- Note : l'utilisation de | | pour agréger une variable avec une chaine de caractères.
- **${…}** permet d'évaluer une expression,
  - Une variable : **${name}**
  - L'attribut d'une variable (de type objet): **${student.gender}**
  - Une expression : **${number * 2}**

```
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        return "greetings";
    }
}
```

greetings.html

```
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
```

EXEMPLE DE TEMPLATE

- 注：使用| |可将变量与字符串进行聚合。
- ${...} 用于计算表达式
  - 一个变量：${name}
  - 对象变量的属性：${student_gender}
  - 一个表达式：${number * 2}

控制台

```
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        return "greetings";
    }
}
```

问候

```
< 体>
    <p th:text= " | 你 好 , ${name}!  | "  />
</body>
```

EXEMPLE D'INSTRUCTIONS

- th:text
- th:each
- th:if
- th:unless
- th:switch
- th:case

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <p th:text="|Hello, ${name}!|" />
     <p th:text="${number}" />
     <p th:text="${number*2}" />

     <ol>
          <li th:each="l: ${li}">
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

```
<tr th:each="student: ${students}">
    <td th:text="${student.id}" />
    <td th:text="${student.name}" />
</tr>
```

```
<td>
    <span th:if="${student.gender} == 'M'" th:text="Male" />
    <span th:unless="${student.gender} == 'M'" th:text="Female" />
</td>
```

```
<td th:switch="${student.gender}">
    <span th:case="'M'" th:text="Male" />
    <span th:case="'F'" th:text="Female" />
</td>
```

EXEMPLE D'INSTRUCTIONS

- 文本
- 每个
- 如果
- 除非
- th:开关
- 胸廓

```
! DOCTYPE HTML>
< html 命名空间 :th= "http://www.thymeleaf.org" ><he-
ad>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <p th:text="|Hello, ${name}!|" />
     <p th:text="${number}" />
     <p th:text="${number*2}" />

     <ol>
          <li th:each= "l: ${li}" >
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

```
<tr th:each= "student: ${students}" >
     <td th:text= "${student.id}" />
     <td th:text= "${student.name}" /></tr>
```

```
<td>
     <span th:if= "${student_gender} == 'M' " th:text= "男性" />
     <span th:除非${student.gender} == 'M' " th:text=" 女性 " /></td>
```

```
<td th:switch= "${student . gender}" >
     <span th:case= 'M' th:text= '男性' />
     <span th:case= 'F' th:text= '女性' /></td>
```

EXEMPLE D'INSTRUCTIONS

- th:each
  - Permet de boucler sur les éléments d'une liste (ou autre)

  - **La boucle répète l'élément sur lequel il est ajouté :**
    - *1er cas :* 1 liste <ol> et plusieurs <li>
    - *2ème cas :* plusieurs <ol> et 1 <li> par <ol>

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <ol>
          <li th:each="l: ${li}">
          <span th:text="${l}" />
          </li>
     </ol>

     <ol th:each="l: ${li}">
          <li>
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

EXEMPLE D'INSTRUCTIONS

- 每个
  - 支持对列表（或其他）中的元素进行循环遍历
  - 循环会重复添加该元素的所在位置：
    - *第一种情况：*1个列表 <ol> 和多个 <li>
    - *第二种情况：*多个 <ol> 和 1 <li> 每 <ol>

```
! DOCTYPE HTML>
< html 命名空间 :th= "http://www.thymeleaf.org" ><he-
ad>
    <title> 开始使用：Web 内容服务 </title>
    <meta http-equiv= "Content-Type" content= " text/html;  charset=UTF-8"  /></head>
< 体>
    <ol>
        <li th:each=  "l: ${li}"  >
        <span th:text=  " ${l}"  / >
        </里
    </

    ol th:each=  "l: ${li}"  >
        <li>
        <span  th:text="${l}"  />
        </li>
    </ol>
</body>
</html>
```

EXEMPLE D'INSTRUCTIONS

- th:each
  - Permet de boucler sur les éléments d'une liste (ou autre)
  - **La boucle répète l'élément sur lequel il est ajouté :**
    - *1er cas :* 1 liste <ol> et plusieurs <li>
    - *2ème cas :* plusieurs <ol> et 1 <li> par <ol>

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <ol>
          <li th:each="l: ${li}">
          <span th:text="${l}" />
          </li>
     </ol>

     <ol th:each="l: ${li}">
          <li>
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

```
1  <!DOCTYPE HTML>
2  <html>
3  <head>
4      <title>Getting Started: Serving Web Content</title>
5      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"
6  </head>
7  <body>
8      <ol>
9          <li>
10             <span >Paris</span>
11         </li>
12         <li>
13             <span >Marseille</span>
14         </li>
15         <li>
16             <span >Lyon</span>
17         </li>
18         <li>
19             <span >Toulouse</span>
20         </li>
21     </ol>
22
23     <ol>
24         <li>
25             <span >Paris</span>
26         </li>
27     </ol>
28
29     <ol>
30         <li>
31             <span >Marseille</span>
32         </li>
33     </ol>
34
35     <ol>
36         <li>
37             <span >Lyon</span>
38         </li>
39     </ol>
40
41     <ol>
42         <li>
43             <span >Toulouse</span>
44         </li>
45     </ol>
46 </body>
47 </html>
```

EXEMPLE D'INSTRUCTIONS

- 每个
  - 支持对列表（或其他）中的元素进行循环遍历
  - 循环会重复添加该元素的所在位置：
    - *第一种情况：* 1个列表 <ol> 和多个 <li>
    - *第二种情况：* 多个 <ol> 和 1 <li> 每 <ol>

```
! DOCTYPE HTML>
< html 命名空间 :th= "http://www.thymeleaf.org" ><he-
ad>
    <title> 开始使用：Web 内容服务 </title>
    <meta http-equiv= "Content-Type" content= " text/html;  charset=UTF-8"  />></head>
< 体>
    <ol>
        <li th:each=  "l: ${li}" ><span
        th:text=  " ${l}" / ></li>
    </

    <ol th:each= "l: ${li}" ><li>
        <span th:text="${l}" />
        </li>
    </ol>
</body>
</html>
```

```
<!DOCTYPE HTML>
<html>
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"
</head>
<body>
    <ol>
        <li>
            <span >Paris</span>
        </li>
        <li>
            <span >Marseille</span>
        </li>
        <li>
            <span >Lyon</span>
        </li>
        <li>
            <span >Toulouse</span>
        </li>
    </ol>

    <ol>
        <li>
            <span >Paris</span>
        </li>
    </ol>

    <ol>
        <li>
            <span >Marseille</span>
        </li>
    </ol>

    <ol>
        <li>
            <span >Lyon</span>
        </li>
    </ol>

    <ol>
        <li>
            <span >Toulouse</span>
        </li>
    </ol>
</body>
</html>
```

EXEMPLE D'INSTRUCTIONS

- th:if
- th:unless

- Ils permettent d'afficher **l'élément auquel elle est appliquée** si une variable est vraie (resp. Fausse).

```
<span th:if="${footer}" th:text="FOOOOOTER" />
<span th:unless="${footer}" th:text="|No Footer|" />
```

```
<div th:if="${footer}">
    <footer>Pied de page footer </footer>
</div>

<div th:unless="${footer}">
    <div class="footer">Pied de page div class footer</div>
</div>
```

EXEMPLE D'INSTRUCTIONS

- 如果
- 除非
- 当变量为真（或假）时，可显示该元素所对应的内容。

```
<span th:if= “${footer}” th:text= “ FOOOOOTER” />
<span th:除非= “${footer}” th:text= “|无页脚|” />
```

```
<div th:if= “ ${footer}” >
    <页脚> 页脚 </页脚></div>

<div th:除非= “ ${footer}” >
    <div class= “footer” >页脚 div class footer</div></div>
```

EXEMPLE D'INSTRUCTIONS

- th:switch
- th:case

```
<td>
    <span th:if="${student.gender} == 'M'" th:text="Male" />
    <span th:unless="${student.gender} == 'M'" th:text="Female" />
</td>
```

```
<td th:switch="${student.gender}">
    <span th:case="'M'" th:text="Male" />
    <span th:case="'F'" th:text="Female" />
</td>
```

EXEMPLE D'INSTRUCTIONS

- th:开关
- 胸廓

```
<td>
    <span th:if="${student_gender} == 'M'" th:text="男性" />
    <span th:除非${student.gender} == 'M' " th:text="女性 " /></td>
```

```
<td th:switch="${student . gender}" >
    <span th:case='M' th:text='男性' />
    <span th:case='F' th:text='女性' /></td>
```

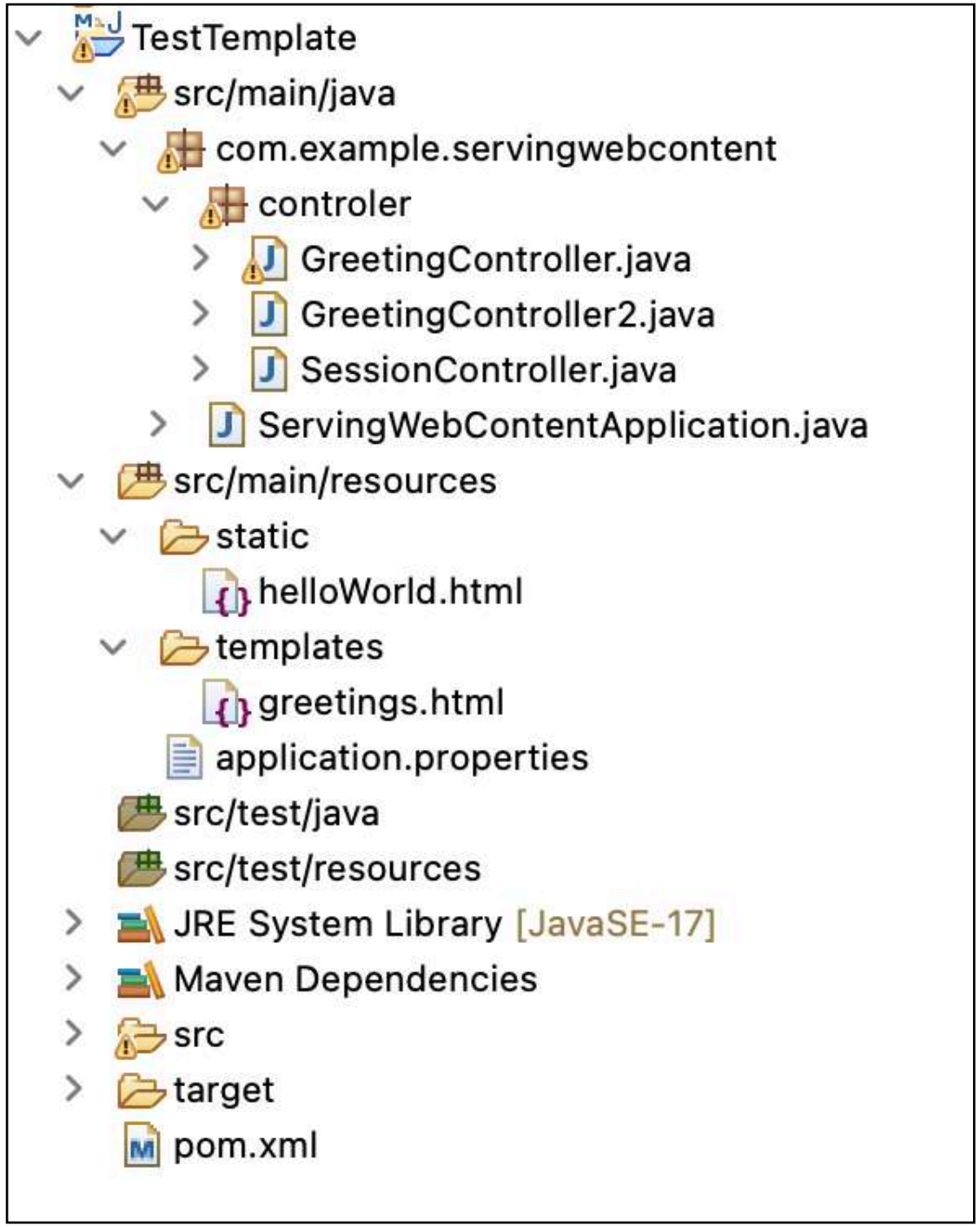
ARCHITECTURE

- Par défaut (avec SpringBoot), les fichiers templates sont placés dans le dossier :
  `src/main/resources/templates`

- Les templates sont des fichiers HTML.

- D'autres pages HTML (sans instructions) sont placées dans le dossier :
  `src/main/resources/static`

- `Src/main/resources` contient également le fichier de configuration
  `application.properties`



ARCHITECTURE

- **默认情况下（使用SpringBoot），模板文件会放置在以下目录中：**src/main/resources/templates

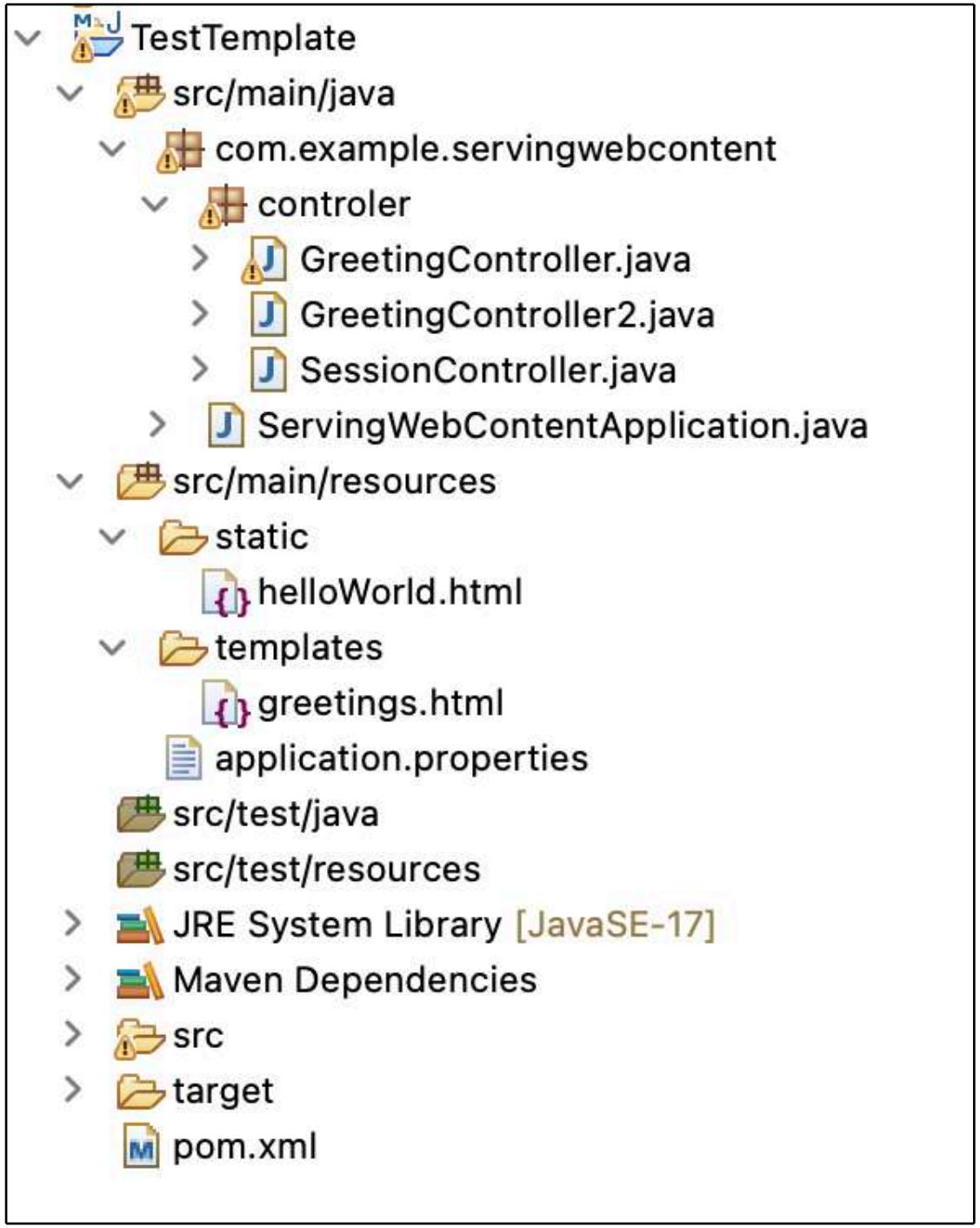
- **模板是HTML文件。**

- **其他无说明的HTML页面被放置在以下文件夹中：**

  源代码资源静态

- Src/main/resources**也包含配置文件**

  应用程序配置



APPELLE FICHIERS, ROUTES

- Une méthode affiche un **template** avec:

  `return "nomTemplateFile";`

- Une méthode affiche une **page static** (donc sans évaluer les instructions) avec:

  `return redirect:/fichierHTML.html`

- **`redirect` peut être utiliser avec une route,** et donc appelle une autre méthode du contrôleur.

```java
@Controller
public class GreetingController2 {


    @GetMapping("/hw")
    public String hw() {
        return "redirect:/helloWorld.html";
    }


    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        model.addAttribute("number", 10);

        ArrayList<Integer> li = new ArrayList<Integer>();
        li.add(23);
        li.add(43);
        li.add(35);
        model.addAttribute("li",li);

        model.addAttribute("footer",false);

        return "greetings";
    }
}
```

APPELLE FICHIERS, ROUTES

- 一种方法会显示一个包含以下内容的模板：

  `return "nomTemplateFile";`

- 该方法会显示一个静态页面（即不执行任何指令），其内容包括：

  返回重定向:/fichierHTML .html

- 重定向可以与路由配合使用，从而调用控制器的其他方法。

```
控制台
public class GreetingController2 {


      GET( “hw” )
      public String hw() {

            返回 “重定向:/helloWorld.html” ; }


      @GetMapping("/greeting")
      public String greeting(
                  @RequestParam(name="name", defaultValue="World") String name,
                  Model model) {
            model.addAttribute("name", name);
            model.addAttribute("number", 10);

            ArrayList<Integer> li = new ArrayList<Integer>();
            li.add(23);
            li.add(43);
            li.add(35);
            model.addAttribute("li",li);

            model.addAttribute("footer",false);

            return "greetings";
      }
}
```

REMARQUE

- Les méthodes associées à une route peuvent combiner @PathVariable, @RequestParam et model…

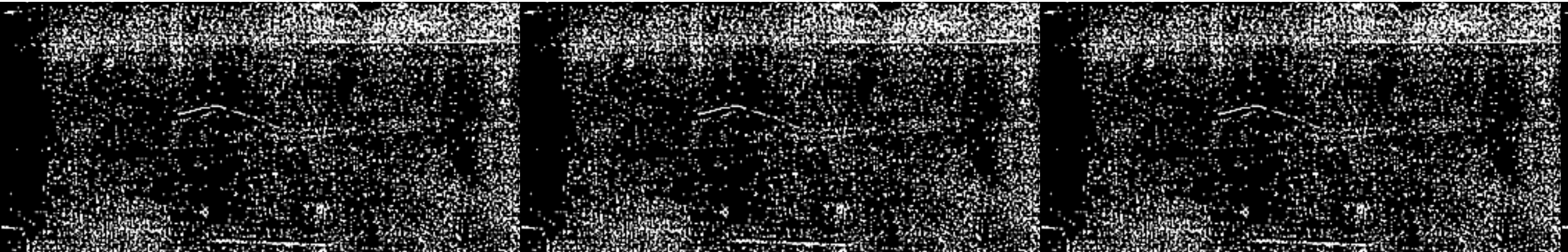
```
    @RequestMapping("/login")
    public String login(
          @RequestParam(name="name", required=true) String name,
          @RequestParam(name="givenname", required=true) String givenName,
           Model model) {
        session.setAttribute("sName", name);
        session.setAttribute("sGivenName", givenName);
        return "index";
    }
```

REMARQUE

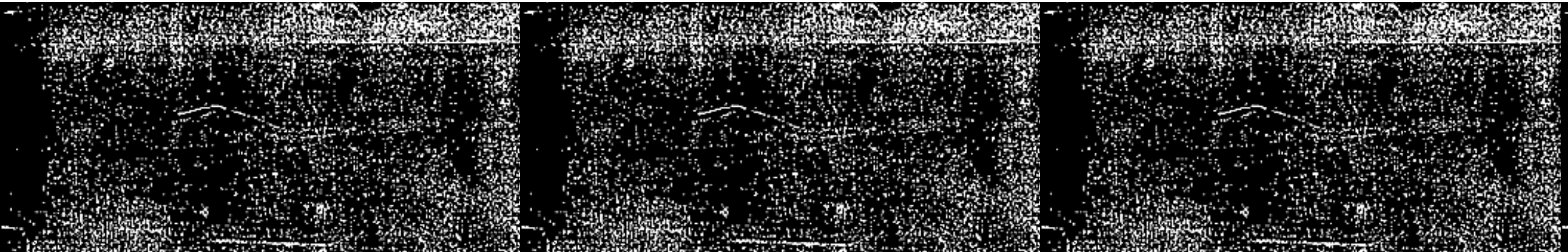
- 与路由相关的方法可同时整合@PathVariable、@RequestParam及model等参数类型……

```
    @RequestMapping("/login")
    public String login(
            @RequestParam(name="name", required=true) String name,
            @RequestParam(name="givenname", required=true) String givenName,
            Model model) {
        session.setAttribute("sName", name);
        session.setAttribute("sGivenName", givenName);
        return 索引    ;
    }
```

SESSIONS



SESSIONS



INTRODUCTION

- Après l'envoi de la réponse HTTP, client et serveur ne sont plus connectés.
  - Il n'est donc pas possible d'envoyer des données entre pages du site. ex: panier, information de connection …

- L'objectif des **sessions** est d'enregistrer en mémoire des données qui vont être accessibles sur plusieurs pages, pendant une durée donnée.
- Ces informations proviennent de choix / sélection de l'utilisateur lors de sa visite du site.

INTRODUCTION

- 在发送HTTP响应后，客户端与服务器将断开连接。
  - 因此无法在网站页面之间传输数据，例如：购物车、登录信息等。
- 这些会话的目的是将数据存储在内存中，以便在指定时间内跨多个页面访问。
- 这些信息来源于用户访问网站时的主动选择。

SESSIONS

- Plusieurs alternatives techniques existent pour stocker ces données :
  - En mémoire sur le serveur
  - Cookie côté client
  - Base de données
  - …

- Ces sessions peuvent être limitées dans le temps.
  - Le *timeout* des session peut être fixé dans le fichier de configurations `application.properties`

```
server.servlet.session.timeout=30m
```

SESSIONS

- 存在多种技术方案可用于存储这些数据：
  - 服务器上的内存
  - 客户端Cookie
  - 数据库
  - …
- 这些会议可能会受到时间限制。
  - 会话*超时*可在配置文件`application.properties`中设置

    服务器。servlet。会话。超时=30分钟

SESSION HTTP PAR DÉFAUT : EN MÉMOIRE

- SpringBoot utilise Spring Session pour gérer des sessions HTTP.
- Par défaut, SpringBoot stocke les données de session en mémoire en utilisant les `HttpSession` de Tomcat.

- Les sessions sont accessibles directement en passant un attribut supplémentaire aux méthodes associées aux routes.

```
@RequestMapping("/setSession")
public String setSession(HttpSession session) {
    session.setAttribute("user", "JohnDoe");
    return "Session attribute set";
}

@RequestMapping("/getSession")
public String getSession(HttpSession session) {
    return "User: " + session.getAttribute("user");
}
```

SESSION HTTP PAR DÉFAUT : EN MÉMOIRE

- SpringBoot 使用 Spring Session 来管理HTTP会话。
- 默认情况下，SpringBoot会使用Tomcat的HttpSession将会话数据存储在内存中。

- 用户可通过为路由相关方法添加额外属性，直接访问这些会话。

```
请求返回（“/setSession”）
public String setSession(HttpSession session) {
    session.setAttribute("user", "JohnDoe");
    return "Session attribute set";
}

 “/获取会话信息”
public String getSession(HttpSession session) {
    return "User: " + session.getAttribute("user");
}
```

UTILISATION DES SESSIONS

- Une variable de session est une sorte de **dictionnaire** contenant des données identifiées par une clé (de type string).
- `session.setAttribute(String, Object);`

  Attache un objet à cette session, en utilisant le nom spécifié. Si un objet du même nom est déjà lié à la session, l'objet est remplacé.
- `session.removeAttribute(String);`

  Supprime de cette session l'objet lié au nom spécifié. Si la session n'a pas d'objet lié au nom spécifié, cette méthode ne fait rien.
- `session.getAttribute(String)`

  Renvoie l'objet lié au nom spécifié dans cette session, ou **null** si aucun objet n'est lié à ce nom.

UTILISATION DES SESSIONS

- 会话变量是一种存储数据的字典
  通过字符串形式的键进行标识。
- `session.setAttribute(String, Object);`

  使用指定名称将对象绑定至当前会话。若该名称已存在关联对象，则会替换原有对象。

- `session.removeAttribute(String);`

  从当前会话中删除指定名称关联的对象。若该会话中不存在与指定名称关联的对象，此方法将不执行任何操作。

- 会话.Attribute(字符串)

  返回与当前会话中指定名称关联的对象，若无关联对象则返回null。

EXEMPLE D'UTILISATION DES SESSIONS

```
@RequestMapping("/getSession")
public String getSession(HttpSession session) {

     String elts = "";
    Enumeration<String> enume = session.getAttributeNames();
    while (enume.hasMoreElements()) {
         String key = enume.nextElement();
         elts += key + session.getAttribute(key) + " - ";
    }

 return elts;
}
```

EXEMPLE D'UTILISATION DES SESSIONS

```
 "/获取会话信息"
public String getSession(HttpSession session) {

     字符串元素= "" ;
    Enumeration<String> enume = session.getAttributeNames();
    while (enume.hasMoreElements()) {
         String key = enume.nextElement();
         elts += key + session.getAttribute(key) + " - ";
    }

return elts;
}
```

REMARQUE

- Les méthodes associées à une route peuvent combiner HttpSession, @PathVariable, @RequestParam et template…

```
@RequestMapping("/login")
public String login(
        @RequestParam(name="name", required=true) String name,
        @RequestParam(name="givenname", required=true) String givenName,
        HttpSession session,
        Model model) {
    session.setAttribute("sName", name);
    session.setAttribute("sGivenName", givenName);
    return "index";
}
```

REMARQUE

- 与路由相关的方法可以组合使用HttpSession、@PathVariable、@RequestParam和模板.....

```
"/登录"
公共字符串登录(
        @RequestParam(name= "name" , required=true) 字符串name,
        @RequestParam(name= "givenname" , required=true) 字符串givenName, HttpSession session,
        模型模型) {
    会话. setAttribute( "sName" , name);
    会话。setAttribute( "sGivenName" , givenName);
    return 索引 ;
}
```

TEMPLATE ET SESSION

- La variable de session est directement accessible depuis le template.

```
<ol>
    <li th:each="l: ${session}">
     <span th:text="${l}" />
    </li>
</ol>

<span th:text="${session.user}">
```

1. user1=toto
2. user2=titi
3. user=JohnDoe
4. user3=teuteu

JohnDoe

TEMPLATE ET SESSION

- 会话变量可直接从模板中访问。

```
<ol>
    <li th:每个= " l: ${session}" >
     < span th:text= "${l}" /></li>
</

<span th:text= "${session.user}" >
```

1. user1=toto
2. user2=titi
3. user=JohnDoe
4. user3=teuteu

JohnDoe